

FREE STARTER KIT

PRACTICAL. HONEST. RECEIPTS-DRIVEN.

I CALL BS



AI VIBE CODING MYTHS DISPELLED

A Realistic Guide to Building Real Apps with Claude Code + Cursor

✓ PRACTICAL WORKFLOWS
✓ PRODUCTION-READY APPS

✓ REAL PROJECTS
✓ RESULTS THAT STICK

V. JIMALE RIDGEWAY

WHAT THIS FREE KIT IS

This is the honest version of “build an app with AI.” No fairy tales. No thirty-minute miracles. The myth-busting, the mindset, and just enough of the toolkit to prove it’s real.

If it lands, the full guide is the next step – thirteen chapters, the complete gate, the prompt library, the case study, the full glossaries, and the closing argument. Everything here is yours to copy, edit, and keep.

CONTENTS

01	Myth vs Reality	3	–	Where to Go from Here	21
02	AI Is Not Smarter Than You	7	–	The Pocket Glossary	22
03	Three Lessons from the Receipts	13	–	Proof Your Work	26
04	The Literacy Gate	15	–	The Smoke Test	27
05	The Toolkit	18	–	Before You Ship	29

READ ME

The numbers here come from my own transcript analysis while building two real apps – not universal industry statistics. Practical education, not legal, security, or financial advice.

MYTH VS REALITY

THE CLAIM

“Influencers say you can type a simple prompt and build an entire app concept in five to thirty minutes. Sign up with their link.”



I believed a softer version of that story longer than I should have. Then I built real apps with Claude Code and Cursor – and for anything with auth, data, and payments, that thirty-minute story fell apart under the weight of my own chat history.

MY RECEIPTS

I analyzed my Cursor agent transcripts while building two apps with production-style stacks: a web platform and a React Native / Expo mobile app. (App names kept private on purpose.)

Cursor agent transcripts	
2 apps - 221 chats	

Chat sessions	221
Prompts I typed	3,302
AI tool calls	50,669
File edits by the AI	16,612
Terminal commands	4,338
Prompts that looked like corrections / frustration	about 544
	(roughly 1 in 6)
Chats where I pasted screenshots	132 of 221 (60%)
Longest single chat	501 prompts
Median chat	4 prompts / ~28 min
Prompts between midnight and 5 AM	278

WEB PLATFORM + REACT NATIVE / EXPO	

Almost none of those prompts were “build me an app.” Most were: **fix this, don’t touch that, put it back, still broken, make the cards smaller, the email never arrived, logout still doesn’t work.**

WHAT THE DEMOS NEVER SHOW

THE LOGOUT BUTTON THAT WOULDN'T DIE.

I reported it across multiple chats. One “fix” made it clickable — it still didn’t log out. About twelve days later it was still broken. The real cause was subtle: tokens deleted from one storage layer were resurrected from another. A dedicated audit then found eighteen more issues in code I thought was done.

THE “SMALL IMPROVEMENT” THAT BROKE PAYMENTS.

Checkout worked. I asked to speed it up. Payments failed. A page went blank. I eventually typed the line I now treat as law:

IF IT AIN'T BROKE, DON'T FIX IT.

THE REAL JOB TITLE

You're not prompting apps into existence.

You're the product manager, tech lead, and QA department for a very fast contractor who forgets context between sessions, over-scopes by default, sounds confident when wrong — and can still 10x you when managed well.

THE THESIS

Claude Code and Cursor did not build my apps in thirty minutes. They made it possible for one person to build them *at all* — at the cost of thousands of prompts, constant verification, and learning to manage AI like a contractor with amnesia.

TATTOO THIS ON YOUR MONITOR

**SPEC → BRANCH → SMALL
CHANGE → CLICK TEST →
COMMIT → NEXT.**

SHORTHAND *Additive. Verify. Commit. Then next.*

AI IS NOT SMARTER THAN YOU

THE FERRY PROBLEM

A friend of mine was planning his first trip to Martha's Vineyard. He asked ChatGPT for advice on where to stay – which town was closest to the ferry terminal.

ChatGPT told him **Falmouth.**

Falmouth is not on the island. It's on the mainland, across the water. If he'd followed that advice, he'd have spent his week commuting by ferry every morning and back every evening – a forty-five-minute trip each way. The last ferry leaves at 9 PM. He'd have missed every evening event. His vacation would have been a logistical nightmare.

Here's the thing: ChatGPT wasn't technically wrong. If you're standing on the mainland, Falmouth *is* the closest town to the ferry. The AI had the correct fact. It had no idea what my friend actually needed.

He didn't ask, "Where should I stay on the island?" — the question he should have asked. He asked, "What is the closest town to the ferry?" The AI answered exactly what he asked. The right answer was Oak Bluffs: you get on in Falmouth, and you get off in Oak Bluffs, on the island.

**THAT'S NOT A
KNOWLEDGE GAP.
THAT'S AN EXPERIENCE
GAP.**

The AI has never missed a ferry. It has never stood in a parking lot at 10 PM watching the last boat leave without it.

THE LADDER

Here’s a way to see it clearly. Seven rungs, from the ground up.

WISDOM	The top. The ability to make sound judgments in situations that don’t have a rulebook. It answers <i>what’s the right thing to do and why</i> .
SMART	Speed. The rate at which you acquire and apply knowledge. Not a rung of its own — a measure of how fast you move between the others.
KNOWLEDGE	Information you’ve internalized and can act on. It answers <i>how</i> .
INFORMATION	WHERE AI SITS Data given context. “The patient’s temperature is 98.6. It’s cloudy outside. The room is 72 degrees.” Now the numbers mean something.
DATA	WHERE AI SITS Raw material. “98.6.” “Cloudy.” “72.” On its own, meaningless.
INTUITION	The gut. Your brain has been quietly noticing patterns your whole life, and every so often it hands you a feeling without showing its work. A firefighter knows the floor is about to collapse before he can explain why.
INSTINCT	The floor. A fixed, hardwired response. A baby grips your finger. You pull your hand off a hot stove before you feel the pain.

↑ FROM THE GROUND UP

WHERE AI ACTUALLY SITS

AI is stuck on rung two. It has Data and Information. That's it.

It has no instinct — it wasn't born, it has nothing to preserve. No intuition — no gut, no lived pattern. No knowledge in the way you do — it has retrieval, not internalization. It can tell you how to ride a bike. It has never felt balance.

*A calculator is faster at math than a Nobel laureate. We don't call the calculator smarter. We call it **fast**.*

THE SLOP, LIVE

I was working on this book. I'd just explained my thesis to a group — “AI is not smarter than you. It has more information. What you get out is only as good as what you put in.”

I asked the AI to summarize my point back to me, to test whether it understood. The response was polished. Grammatically correct. It said:

AI “AI is not smarter than you. It is more well-read than you. But reading 10,000 books doesn't make you wise; it makes you a librarian. Your job is not to compete with the librarian's memory. Your job is to be the editor-in-chief who decides which books matter, which chapters are lies, and which pages get thrown in the trash.”

It sounded good. It was hollow.

Reading 10,000 books would make a person wiser, more likely than not. A librarian is an administrative role, not a scholarly one. And it wasn't my voice. I would never say that. It sounded like AI.

So I called it out. I said:

Me "That sounds like it was written by AI. It's not the way I would speak or write."

The AI immediately apologized. It agreed with me. It said:

AI "You are absolutely right. I apologize. That was AI-slop."

Then it generated another polished, grammatically correct, thematically adjacent summary — with a different metaphor. Just as hollow. Just as wrong. Just as not my voice.

It didn't correct itself. It didn't learn. It gave me a different flavor of the same empty fluency.

That is hallucination in miniature. AI does not reliably know when it's wrong. It has no internal truth-checker. It's a next-token prediction machine. When it fills a gap it doesn't know it has, it does so confidently. And when you correct it, it agrees with you just as confidently — and fills the gap again, the same way, with a different set of words.

THE MAP AND THE TERRITORY

Think of the whole of human knowledge as a vast, ancient city.

AI is a map of that city. An extraordinarily detailed one. It has memorized every street, every building, every restaurant menu. It can give you turn-by-turn directions faster than any native could.

But a map has never walked the streets. It has never smelled the bread from the bakery. It has never gotten lost and found something beautiful by accident.

You are the native. You don't have the whole city memorized. You stumble on some of the names. But you know which bench to take your friend to when they're crying. You feel a storm coming before the sky turns.

The map is useful. Use it. But never confuse the map for the ground you walk on.

YOU ARE THE TERRITORY.

And you are the only one who can tell the map where to go.

THREE LESSONS FROM THE RECEIPTS

The full guide has ten. Here are three that show the range.

LESSON

01

BE CAREFUL WHAT YOU ASK FOR

As you go, determine which files and which code are sacred, and force the agent to ask before touching them. Be sure to keep additive code as separate as possible so you don't break or disturb things that already work and have been tested.

Create rules to this effect and be sure to reinforce them as you move along. You know, because of the amnesia thing.

LESSON

02

WHAT YOU SEE IS NOT NECESSARILY WHAT YOU GET

Your app may behave one way for you and completely differently for a new user. Your account as the creator and dev may already be stocked with data. Everything may look good on your end — but a new user may hit a path you never tested.

A screen that's unavailable, a button that is greyed out, data that never loads. Create new user accounts to test from, and enlist testers before you ship.

LESSON

03

CHAT AMNESIA IS VERY REAL

You may remember what task you completed yesterday, but chances are your AI model does not. To AI, every new chat is a blank slate. AI only knows your notes, your rule files, and your commit history.

Therefore, you have to put everything important in the repo. **Do not depend on the chat.**

THE LITERACY GATE

The second myth, right after the thirty-minute one: that you need zero technical knowledge to build a real app. Here's the honest version.

WHAT'S TRUE

You don't need a CS degree.
You don't need to write code
from a blank page.

WHAT'S ALSO TRUE

You can't catch a mistake you don't know exists — especially when you don't have the vocabulary to call it by name. A database rule that silently leaks one user's data to another throws no error. It's invisible to "I don't need to know anything," and only visible to someone with enough literacy to think to check.

DON'T CAP YOUR API KEYS?

One gets exposed, a bot finds it in hours, and it's drained while you sleep. This happened to me — a leaked key got run up in a single day and got my account suspended. I only got off easy because I'd capped it at fifty dollars.

DON'T KNOW WHAT ROW-LEVEL SECURITY IS?

You can ship an app where every user reads every other user's private data — and never know until a customer tells you.

DON'T KNOW "LOGGED OUT" HAS TO MEAN EVERYWHERE?

A user hits logout, hands off their laptop, and is still signed in.

AI DIDN'T REMOVE THE NEED TO UNDERSTAND YOUR APP. IT CHANGED THE JOB FROM AUTHOR TO EDITOR.

The bar isn't mastery — it's literacy. Enough to catch the AI when it's confidently wrong. **That's a weekend, not a degree.**

THE 8-STEP PREP LIST, BEFORE YOU BUILD ANYTHING REAL

1**The mental model**

Frontend, backend, database, and how they talk.

2**Read code, don't write it from scratch**

Enough to skim a diff and say “that’s not right.”

3**Learn Git**

Your undo button. Branch before every risky prompt.

4**Read errors, slowly**

The answer is usually in the error text.

5**Five terminal commands**

That’s most of what you’ll ever type.

6**Pick one stack**

And stop tool-hopping.

7**Secrets, .env, and billing caps**

Keys are passwords. Never committed. Always capped.

8**Build something tiny first**

Before you build the big thing.

TWO PROMPTS TO USE AI AS A TUTOR

EXPLAIN IT AT MY LEVEL**COPY**

Explain [concept] to me like I have never coded before. One real-world analogy. Define jargon immediately. Give me one sentence I could say to a friend to explain it back.

REVEAL MY BLIND SPOTS**COPY**

Based on what I’ve told you about my project, what are the top 3 things a beginner in my position is most likely to not know they don’t know? Be specific to my stack, not generic.

THE TOOLKIT

THE PRIME DIRECTIVE

PASTE OFTEN

COPY

PRIME DIRECTIVE – read twice:

- Existing behavior must not break.
- Prefer additive changes and new files.
- Do not refactor unrelated code.
- If you think a core file must change, STOP and ask first.
- Do not change brand colors/tokens unless I ask.
- If you can run a command yourself, do it – except deleting data, destructive database changes, or anything in production. Ask me first on those.

As always: we are adding on, not taking away. Don't change anything that already works. Keep new work as separate as possible.

TWO PROMPTS TO STEAL

UI ONLY

[COPY](#)

```
Change [SCREEN/COMPONENT] only.
Problem: [what looks wrong]
Desired: [what it should look like]
Screenshot: [attach]
Rules:
- Use the existing design tokens / colors —
do not invent new ones.
- Do not rewrite the screen or the shell.
- Touch only the component(s) needed for
this change.
- Show me the list of files you changed.
```

BUG + EVIDENCE

[COPY](#)

```
Bug in [AREA].
Expected: [what should happen]
Actual: [what happens instead]
Steps to reproduce: [1, 2, 3]
Error text: [paste the full error]
Do:
- Find the root cause only. Do not refactor
around it.
- Propose the smallest fix.
- Give me a retest script so I can confirm
it.
```

POCKET GLOSSARY

The words you'll hit first. The full dictionary is in the paid edition.

Vibe coding — Building software by describing intent to an AI agent that edits files and runs commands. Powerful. Not magic.

Prompt — Your instruction. Vague in, vague (or dangerous) out.

Agent — AI that uses tools: read, edit, run commands — not just chat.

Repo — Your project folder tracked by Git, usually also on GitHub.

Commit — A saved snapshot with a message.

Branch — A parallel timeline. Risky AI work belongs here. main stays known-good.

Environment variables / **.env** — Secrets and config. Never commit real secrets.

Deploy — Putting the app where others can use it.

UI / UX — How it looks / how it feels.

THE SMOKE TEST

Run this after any non-trivial AI change. **Done means you clicked it – not that the agent said “fixed.”**

UNIVERSAL, EVERY TIME

- ☐ App boots / localhost loads
- ☐ Can log in
- ☐ Can log out and stay logged out after reload
- ☐ Main / home screen loads
- ☐ The thing I changed works once
- ☐ One nearby feature still works
- ☐ No secrets in the git diff

If you touched payments

Test-mode purchase completes, and the webhook or server log shows success. **UI banners lie.**

If you touched email

The message arrives in a real inbox – not just an API returning 200.

If you touched uploads

The file lands in the bucket and actually renders in the UI.

If you touched an editor / admin screen

Save works, and reload shows the changes are still there.

WHERE TO GO FROM HERE

You just got the myth-busting and the mindset. The full edition is where the *how* lives.

I Call BS: AI Vibe Coding Myths Dispelled – the complete guide (\$17) includes:

- **All thirteen chapters** – from the myths and the literacy gate to the daily prompting process, the proof gates, and life after launch.
- **The full gate** – Proof Before Promises and Knowing When It’s Safe to Open the Gate. The smoke test, adversarial review, the design pass, the security pass, the legal pass, the pre-tester checklist, and the prompt I use to run the whole thing.
- **The pivot chapter** – Expect the Unexpected. The thing you built may not be the thing you end up sharing. And that’s usually a sign you were paying attention.
- **The sharing chapter** – Yay! You’ve Built the App. Now What? Marketing on the same discipline, without letting it rewrite the app.
- **The closing chapter** – the recap, the terms, the quiz, and the argument the whole book has been building toward: due diligence over instant gratification.
- **The Toolkit** – smoke test, Before I Accept “Done,” pre-flight, sacred files, the BETA tester and pre-test-audience checklists, and the full “Proof Your Work” list.
- **The full prompt swipe file** – additive features, bug-with-evidence, adversarial review, restore, and more.
- **Three glossaries** – the Complete Glossary, the Visual Glossary, and Glossary Extra.
- **Five add-ons** – UI & UX, skills and plugins, setting up rules, a worked example with before/after prompts, and the plumbing.

You bring the depth. It brings the breadth. Neither one ships alone.

GET THE FULL GUIDE

[SALES LINK]

\$17

THE POCKET GLOSSARY

The words you'll actually need in week one

You don't need every technical term to build with AI. You need enough to describe what you want — and to name what you're trying to protect. This is that short list: the two big ideas worth genuinely understanding, and the handful of terms you'll trip over first.

The full glossary at the back of the book is for the days after that. When a term shows up in an error you don't recognize. When you can picture the thing you want but can't name it. When you need to tell the AI exactly what to touch — and exactly what not to.

This is the week-one version.

UI & UX — WORTH PULLING APART

Two letters apart, and everybody smushes them together. They're not the same thing, and the difference will save you real grief.

UI

USER INTERFACE

What the app looks like: buttons, cards, colors, spacing, layout. The pixels on the screen.

UX

USER EXPERIENCE

How the app feels to use: the path, the friction, whether the thing you tapped did what you expected. UI is the paint job. UX is whether the car actually drives.

UX is the one that bites you, because a screen can be gorgeous and still be broken. A logout button that looks perfect but leaves you logged in isn't a UI problem – it's a UX lie. Save that doesn't save. Onboarding that loops you back to the start. **Your users never see your code; they only feel the UX.**

WHY THIS MATTERS FOR YOUR PROMPTS

Most bad UI prompts aren't bad because you can't code – they're bad because you don't have the word. If you can't name the thing you want, you get a redesign. Name the part, and the AI changes that part. Can't name it, and it guesses – and guessing is how “move this one button” turns into a whole page rewritten.

And the names carry behavior, not just looks:

SAY THIS	IT MEANS
Radio vs. checkbox	Pick one vs. pick many.
Padding vs. margin	Space inside a box vs. space between boxes.
Modal vs. drawer vs. sheet	Blocks the whole page vs. slides in from the side vs. slides up from the bottom.

Say the wrong one and you don't get an ugly result – **you get the wrong thing.**

Every element, in plain English – plus a “describe it, name it” lookup for when you can picture the thing but can't name it – lives in The Complete Glossary at the back of the book.

BACK END – THE ENGINE ROOM

If the front end is what users see, the back end is everything they don't: the server that runs your logic, the database that remembers everything, the APIs that carry data back and forth, and the auth that decides who gets in. Nobody ever looks at it. They only feel it – when it works, and especially when it doesn't.

Here's what makes the back end dangerous in a way the front end isn't: **front-end bugs are loud; back-end bugs are silent.** A broken button, you see instantly. A database rule that quietly lets one user read another user's data throws no error and shows no red screen – it just sits there leaking until it's a headline. The scariest bugs in this whole book live back here, and none of them announce themselves.

A few pieces worth pulling apart, because people blur them:

DATABASE VS. CACHE

The database is the source of truth – the thing that actually remembers. A cache is a fast copy of it. When they disagree, you get “I saved it but it's gone after reload” – the screen trusted a stale copy.

AUTH VS. SESSION VS. TOKEN

Authentication is proving who you are (login). A session is the app remembering you're logged in. A token is the little proof it stores to do that. This trio is why logout is trickier than it looks: if a token survives in even one storage layer, the user is still logged in. Logout has to clear the proof everywhere.

RLS (ROW LEVEL SECURITY)

The database rule for who can read or write which rows. Right, and users see only their own data. Wrong, and you either leak everyone's or nothing saves – with no error to tell you which.

WHY THIS MATTERS FOR YOUR PROMPTS

Back here, the words are how you protect things. You can't tell the AI "don't touch auth," "check the RLS on that table," or "this is a sacred payment flow – propose a diff first" if you don't have those words. Naming the front-end part gets you the right look. Naming the back-end part keeps you from shipping a leak. **Higher stakes, same rule.**

The full server-side vocabulary – data, auth, APIs, deploys, and the rest, in plain English, plus a "symptom, what's probably wrong" table – is in The Complete Glossary at the back of the book.

YOU'LL HIT THESE IN WEEK ONE

Quick definitions for the terms that show up first. Don't memorize them – just recognize them when they land in an error or a prompt.

Frontend What users see	Backend Server, database, APIs	API How systems talk to each other
Database Where the data lives	Schema The shape of the database	Migration A versioned change to the database
CRUD Create, Read, Update, Delete	State What the app knows right now	Cache A fast stored copy
Component A reusable UI building block	Route A screen or URL	TypeScript JavaScript with types
Lint Automated style / error check	Build Compiling the code into a runnable app	Stack Your tech set
SDK A vendor's toolkit	Webhook A server callback – one service pinging another	Deploy Putting the app where others can use it

Need the full dictionary? It's all in The Complete Glossary at the back of the book.

PROOF YOUR WORK

This book keeps telling you to double-check everything. Here's the starter toolkit to actually do it – work down the list in order.

1

Prove it yourself. Run the smoke test (boot, log in, log out, reload, confirm you stayed out, check what you changed + a nearby thing). Then try to break it: double-click submit, reload mid-action, log in as a second user and try to see the first user's data. Open your browser's DevTools console and network tab – that's where the truth lives when the screen is lying.

2

Turn on the free safety nets. A linter and type-checking (ESLint + TypeScript), error monitoring in production (Sentry), and secret scanning on your repo (GitHub's is built in). Set them once; they catch mistakes forever.

3

Get a second set of eyes – including a non-human one. Have one AI build it, then point a different AI at the diff: “review only, find bugs and security holes, don't be nice.” Then, for anything important, watch a real person use it without your help.

4

Cap and restrict your keys. Every paid API key gets a spending limit and domain restrictions. (I learned this the \$50 way – see the intro.)

KNOW WHEN TO HIRE A HUMAN

For anything touching payments, sensitive user data, or a regulated space (health, finance, kids' data), a tool is not enough. That's a “pay a real professional” situation – a security review or a lawyer is cheap next to a breach or a lawsuit.

The full “Proof Your Work” resources – the complete tier-by-tier list of tools for self-testing, security, compliance, and accessibility – is in the \$17 full guide. Link: [SALES LINK].

KEEP THIS OPEN WHILE YOU BUILD

THE SMOKE TEST

Run it after any non-trivial AI change. Done means you clicked it – not that the agent said “fixed.”

UNIVERSAL – EVERY TIME

- ☐ App boots / localhost loads
- ☐ Can log in
- ☐ Can log out and stay logged out after reload
- ☐ Main / home screen loads
- ☐ The thing I changed works once
- ☐ One nearby feature still works
- ☐ No secrets in the git diff

THAT LAST ONE

That last one is the one that costs money. Scan the list of files you’re about to save and make sure nothing private is sitting in it – a password, a key, a token. If you’re not sure, stop and look.

IF YOU TOUCHED PAYMENTS

- ☐ Test-mode purchase completes
- ☐ The webhook or server log shows success

The screen saying “Paid” is not proof. The screen lies.
The server tells the truth.

IF YOU TOUCHED UPLOADS

- ☐ The file lands in the bucket
- ☐ The file actually renders in the UI

SHIPPING EXTRAS

WEB

- ☐ main is green locally
- ☐ Env vars set on the host, not just on your laptop
- ☐ Migrations applied to the target database
- ☐ Auth redirect URLs allow your domain
- ☐ Smoke test on the production URL
- ☐ A rollback plan exists

IF YOU TOUCHED EMAIL

- ☐ The message arrives in a real inbox

Not just an API returning 200. The message actually lands. Check spam.

IF YOU TOUCHED AN EDITOR / ADMIN SCREEN

- ☐ Save works
- ☐ Reload – the changes are still there

MOBILE

- ☐ Build succeeds
- ☐ Tested on a real device
- ☐ Cold-start auth behaves correctly
- ☐ Privacy text matches real behavior

DONE MEANS YOU CLICKED IT. NOT THAT THE AGENT SAID “FIXED.”

BEFORE YOU SHIP

A five-line pre-flight. Run it the day you're about to show someone.

THE SHORT LIST

- ☐ Smoke test passes on a non-seeded account – one a stranger would sign up with fresh
 - ☐ Login, logout, and “stays logged out after reload” all work
 - ☐ The one flow you care about works for someone who's never seen it
 - ☐ No secrets in the repo, exports, or chat logs
 - ☐ You know what “done” means – and you can describe it in one sentence
-

IF ANY OF THOSE IS UNCHECKED, DON'T INVITE ANYONE YET.

WHY THE SHORT LIST MATTERS

The first four are about whether the app works for someone who isn't you. You built it. You know where everything is. You can't un-know it. A stranger can't, and they'll hit the wall you don't see anymore.

The last one – describing the app in one sentence – is the real gate. If you can't, the world can't either. If you describe it differently than your testers do, you have a positioning problem, not a product problem. **Fix it now, while the audience is still small.**

WHAT THE FULL GATE ADDS

The short list is the floor. The full gate before real testers is longer, and it runs in order:

ADVERSARIAL REVIEW

A different AI tries to break what the first one built. Design, security, legal, privacy, copy, data, empty states, errors. Find first, fix second.

DESIGN CONTINUITY

Same job, same shell. Progress bars that skip a stage. Call-to-actions leaking between products. The things that look fine and feel wrong.

SECURITY HYGIENE

No secrets in the code, database rules on, least privilege on every key, assume the app is extractable.

LEGAL, PRIVACY, AND OPS

Privacy matches what the app does. Refunds match reality. Email has unsubscribe and a physical address. User content has report, mute, and block. DMCA exists. Production env has real values, not placeholders.

THE PRE-TESTER CHECKLIST

The full go/no-go, product paths, server truth, trust and compliance, engineering hygiene, human last mile.

THE PROMPT I USE TO RUN IT ALL

Review-only until I say fix. Code fixes separated from owner tasks.

That's the whole thing. The complete gate is in the paid guide.

DON'T HOPE STRANGERS WON'T NOTICE. FIX IT, OR NARROW THE BRIEF.

The full pre-tester gate – adversarial review, design continuity, security, legal, ops, the complete checklist, and the gate prompt – is in the \$17 full guide. Link: [\[SALES LINK\]](#).

**YOU BRING THE
DEPTH. IT BRINGS
THE BREADTH.
NEITHER ONE
SHIPS ALONE.**

THE COMPLETE GUIDE

GET THE FULL GUIDE

[SALES LINK]

\$17